

ARAŞTIRMA SORULARI 07 EKİM

1.Değişken nedir? Bir değişkenin değeri ne zaman değişir?

Değişken, bilgisayar programlamasında ve matematikte, içerisinde veri(değer) saklayan ve bu değeri süreç içinde değiştirebilen dinamik bir bellek alanıdır.

Bir değişkenin değeri şu durumlarda değişir:

-Yeni bir atama yapıldığında(programlamada):

kodun ilerleyen aşamalarında değişkene = (atama) operatörü ile yeni bir değer verildiğinde, eski değer silinir ve değişken yeni değeri alır.

-Matematiksel bir işlem uygulandığında:

Bir değişken mevcut değerine ekleme, çıkarma,çarpma veya bölme yapıldığında değeri güncellenir. (örn:sayi= sayi + 1).

-Bir döngü (loop) içinde:

Program belirli bir işlemi tekrar ederken, her adımda (örneğin sayaç değişkenleri) değişkenin değeri otomatik olarak artabilir veya azabilir.

-Fonksiyon veya parametre değişikliklerinde:

matematikte bir Fonksiyona girdi olarak verilen bağımsız değişkenin değeri değiştiğinde, buna bağlı olan bağımlı değişkenin değeri de değişir.

2.Program state / durum kavramı ne anlama gelir?

Şu ana kadar her şey sabitti: veri giriyor, bir sayfa çıkıyor ve hiçbir şey değişmiyordu.

Gerçek uygulamalar ise değişen şeyleri hatırlatır:bir beğeni sayısı, açık bir menü, bir arama kutusundaki metin. Bu hafızaya state(durum) adı verilir.

```
import { useState } from 'react';
```

```
export default function App() {  
  const [count, setCount] = useState(0);  
  ...  
}
```

Tek bir satır, üç parça:

-Count: mevcut değer

-setCount: bunu değiştiren Fonksiyon

-useState(0): başlangıç değeri

Altın kural:asla doğrudan state'e atama yapmayın. count = 5, react'ın izlemediği bir değişkeni değişkeni değiştirir. setCount(5)ise state'i değiştirir ve bileşeni yeniden render eder.Ekranı güncelleyen şey bu yeniden render işlemidir.

3.Array,List ve Dictionary hangi ihtiyaçlarda kullanılır?

1.Array(dizi) ne zaman kullanılır?

-Boyutu kesinlikle değişmeyecekse:

Veri adedi sabit ve baştan biliniyorsa tercih edilir. Örneğin;haftanın günleri (7 gün) veya yılın ayları (12 ay).

-En yüksek performans ve düşük bellek kullanımı gerekiyorsa:

Bellekte ardışık olarak saklandıkları için elemanlara indeks(örn: dizi[3]) ile erişmek çok hızlıdır. Bellek harcaması minimumdur.

-Çok boyutlu matrisler gerekiyorsa: Matematiksel hesaplamalar, grafik işleme veya oyun programlamada (örneğin 2B veya 3B koordinat sistemleri) array yapıları idealdir.

2.List(liste) ne zaman kullanılır?

-Veri sayısı sürekli değişiyorsa:

Eleman ekleme, çıkarma veya araya yeni veri ekleme gibi dinamik işlemlere ihtiyaç duyulduğunda kullanılır.

-Verilerin sırası önemliyse:

Elemanların eklenme sırasına göre dizilmesi ve bu sıranın korunması gerektiğinde tercih edilir.Örneğin; bir alışveriş sepetindeki ürünler veya bir mesajlaşma uygulamasındaki son mesajlar listesi.

-Gelişmiş arama ve sıralama fonksiyonları gerekiyorsa:

Listeler genellikle hazır sıralama(sort), filtreleme(Filter/Find) ve temizleme (clear) gibi built-in metotlarla birlikte gelir, bu da kod yazımını kolaylaştırır.

3.Dictionary(sözlük) ne zaman kullanılır?

-Verilere benzersiz bir anahtar(key) ile hızlıca erişmek gerekiyorsa:

Bir veriyi sırasına göre değil, ona ait özel bir kimlikle(ID, T.C. Kimlik No, Barkod, Plaka, E-posta) çağırmak istiyorsanız en doğru yapıdır.

-Milyonlarca veri içinden anında arama yapmak gerekiyorsa:

Dictionary, arkasındaki "Hash" mekanizması sayesinde veri kümesi ne kadar büyük olursa olsun aranan elemanı neredeyse anında (O(1) zamamda) bulur.

-İlişkili veri çiftleri saklanacaksa: Anahtar-Değer (key-value) mantığıyla çalışan durumlarda kullanılır. Örneğin; ülke kodları ve ülke isimleri ("TR ->"Türkiye"), kullanıcı ID'si ve kullanıcı profil detayları.

Doğru yapıyı seçmek için özet tablo

Özellik	Array (dizi)	List (Liste)	Dictionary (sözlük)
Boyut	Sabittir,sonradan değiştirilemez.	Dinamiktir, büyüüp küçülebilir	Dinamiktir,büyüyüp küçülebilir.
Erişim yöntemi	indeks numarası ile ([0], [1])	indeks numarası ile ([0], [1])	Benzersiz Anahtar (Key) ile
Arama hızı	Sıralı arama yavaştır.	Sıralı arama yavaştır.	Anahtar ile arama çok hızlıdır.
En iyi kullanım senaryosu	Performans kritik, sabit boyutlu veriler	Eleman sayısı belirsiz,sıralı listeler	TC kimlik,ID gibi benzersiz anahtarlı veriler

4.List üzerinde Add, Remove ve Count ne yapar?

Add: Listeye yeni bir eleman ekler. Eklenen genellikle listenin en sonuna yerleştirilir.

Remove: Belirtilen bir elemanı veya belirli bir indeksteki elemanı listeden siler.

Count: Listenin içinde o an toplam kaç tane eleman bulunduğunu veren bir özelliktir.

5.Foreach neyi gezer? Listeyi değiştirir mi?

Foreach döngüsü, bir koleksiyonun (dizi,liste,set veya harita gibi) tüm elemanlarını sırayla gezer.

Genel bir kural olarak, Foreach döngüsü üzerinde gezindiği listenin yapısını(eleman ekleme veya silme) veya listenin orjinal eleman referanslarını doğrudan değiştirmez.

6.Bir değeri Console.WriteLine ile yazdırmak ile bir koleksiyona eklemek arasındaki fark nedir?

Bir değeri Console.WriteLine ile yazdırmak ile bir koleksiyona eklemek arasındaki temel fark, verinin kalıcılığı ve program tarafından tekrar kullanılabilirliğidir.

Console.WriteLine veriyi sadece ekranda gösterip tüketirken, koleksiyona eklemek veriyi bilgisayarın belleğinde (RAM) saklayarak program içinde tekrar tekrar kullanmanızı sağlar.

DETAYLI KARŞILAŞTIRMA:

Özellik:

-Console.WriteLine (Ekran Yazdırma) -Koleksiyona Ekleme (List.Add vb)

Amacı:

-Kullanıcıya bilgi göstermek veya hata ayıklama (debug) yapmak.

-Veriyi daha sonra işlemek, değiştirmek veya saklamak.

Bellek Durumu:

- Veri ekrana yazıldıktan sonra bellekten silinir (kaybolur).
- Veri, program açık olduğu sürece bellekte (RAM) saklanır.

Tekrar Kullanım:

- Program bu veriye bir daha erişemez ve üzerinde işlem yapamaz.
- Kodun herhangi bir yerinde veriye tekrar erişilebilir ve değiştirilebilir.

Etki Alanı:

- Sadece dış dünyaya (konsol ekranına) bir çıktı verir.
- Programın iç mantığını ve veri yapısını günceller.

KOD Örneği:

```
csharp
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        // 1. YÖNTEM: Sadece Console.WriteLine kullanmak
        Console.WriteLine(25);
        // 25 sayısı ekrana yazıldı ve bitti.
        // Programın alt satırlarında bu "25" sayısını alıp 5 ile çarpamayız, çünkü hafızada yok.
```

```
        // 2. YÖNTEM: Koleksiyona eklemek
        List<int> sayilar = new List<int>();
        sayilar.Add(25);
        // 25 sayısı artık "sayilar" isimli listenin içinde, bilgisayarın hafızasında duruyor.

        // Hafızadaki bu veriyi daha sonra istediğimiz gibi kullanabiliriz:
        int yeniDeger = sayilar[0] * 5;
        Console.WriteLine("Listenin ilk elemanının 5 katı: " + yeniDeger);
    }
}
```

```
}
```

Özetle; Console.WriteLine bir çıktı(output) işlemidir, koleksiyona eklemek ise bir veri saklama(storage) işlemidir.

Bir List üzerinde “oluşturma - veri ekleme - veriyi okuma - veriyi silme” adımlarını şema hâline getirin.



KOD Örneği:

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        //1.liste oluşturma
        List marketListesi = new List();
```

```
        //2.listeye veri ekleme

        marketListesi.Add("ekmek");
        marketListesi.Add("süt");
        marketListesi.Add("yumurta");

        Console.WriteLine("listedeki ürünler:");

        // 3.listedeki verileri okuma
        foreach(string urun in marketListesi)
        {
            Console.WriteLine(urun);
```

```
}  
//4.listeden veri silme  
marketListesi.Remove("süt");  
Console.WriteLine("\nsüt silindikten sonra:");  
foreach(string urun in marketListesi)  
{  
    Console.WriteLine(urun);  
}  
Console.ReadLine();  
}
```

}

8.Array, List ve Dictionary için günlük hayattan birer kullanım örneği bulun:

Array(Dizi):

-örnek:

-Haftanın günleri veya bir sinema salonundaki koltuk numaraları.

-Neden Array?:

-Haftanın günlerinin sayısı(7 gündür) ve sırası asla değişmez.Array'lar de hafızada sabit bir boyuta sahiptir ve eleman sayıları sonradan değiştirilemez.Pazartesiye 1.gün, salıya 2.gün diyerek direkt sırasıyla (indeksle) erişebiliriz.

List(Liste):

-örnek:

-Alışveriş listesi veya yapılacaklar (To-Do) listesi.

-Neden List?:

- Markete gitmeden önce listenize yeni maddeler ekleyebilir, aldığınız ürünleri listeden silebilirsiniz.List veri yapıları dinamiktir; ucu açıktır, içeriğe göre büyüyebilir veya küçülebilir.

Dictionary(Sözlük/Map):

-örnek:

-Gerçek bir dil sözlüğü, telefon rehberi veya plaka kodları.

-Neden Dictionary?:

-Bir arkadaşınızı aramak için tüm rehberi baştan sona taramazsınız.“Ahmet” ismini (Anahtar/Key)aratır ve karşısındaki telefon numarasına (Değer/Value)direkt ulaşırsınız.Aynı şekilde “34” koduna aratıp karşılığında “İstanbul” bilgisine ulaşmak tam bir Dictionary mantığıdır.

“Ekran yazdırmak veriyi deęiřtirmek deęildir” cümlesini kendi örneęinizle açıklayın:

-Ekran sadece bir aynadır.Aynada gördüğünüz görüntü deęiřtięinde (örneęin ekrandaki yazıyı silip başka bir řey yazdırdıęında) arkadaki gerçek nesne veya veri etkilenmez.Programlamada print(bakiye) komutu sadece deęeri gösterir, bakiye = bakiye -200 komutu ise veriyi deęiřtirir.